

XS WALLET

HD Wallet + Atomic Swaps Desktop

■ SELF-CUSTODY • ■ LIGHTNING • ■ TAPROOT

Documentação Técnica v0.1.0

Janeiro 2026

Equipe XS Wallet

■ Sumário Executivo

Visão geral do projeto XS Wallet

XS Wallet é uma aplicação desktop self-custody que permite atomic swaps entre Bitcoin, Liquid e Lightning Network usando Taproot e a API Boltz v2 (REST + WebSocket) com reconstrução local de scripts/taptree e enforcement de claim/refund via script-path.

Características Principais

- Carteira HD compatível com BIP39/32/84/85
- Suporte Multi-Chain: Bitcoin, Liquid, Lightning
- Atomic Swaps via Boltz API v2 com verificação local
- Integração Taproot com MuSig2 + fallback script-path
- Auto-custódia total com criptografia Argon2id + AES-256-GCM
- Aplicação Electron com nodes embarcados

Status do Projeto

- Fundação do storage/infra completa (15%)
- Cronograma: 5 semanas até MVP
- Database layer production-ready (SQLite + WAL)
- Node manager com downloads verificáveis
- 8 decisões técnicas críticas documentadas



■ ■ Arquitetura do Sistema

Componentes e fluxo de dados

O XS Wallet é estruturado em múltiplas camadas para garantir separação de responsabilidades, isolamento de falhas e manutenibilidade. O backend roda em processo Node.js separado para crash isolation.

Componentes Principais

- Processo Main (Electron): Janelas, Auto-update, Node Manager, IPC
- Backend Process: Swap Engine, Database, Adapters, Boltz Client
- Frontend React: Onboarding, Dashboard, Swap UI, Settings
- Nodes Embarcados: bitcoind, elementsd, LND (download verificável)

Node Lifecycle: O Node Manager cria datadirs isolados por rede (regtest/testnet/mainnet), executa healthchecks RPC/gRPC, e implementa restart com backoff exponencial em caso de falha. Detecção de versão incompatível de chainstate/wallets ao atualizar nodes, com migração automática quando possível.

■ Banco de Dados

SQLite com WAL mode para concorrência

Tabela	Propósito	Chave Primária
swaps	Estado autoritativo (CAS)	id (TEXT)
swap_events	Trilha de auditoria	seq (AUTOINCREMENT)
swap_ops	Ledger idempotência	(swap_id, op_key)
utxo_reservations	Anti double-spend	(chain, txid, vout)
ln_reservations	Anti duplicação LN	payment_hash_hex
app_config	Configurações	key

Pragmas Críticos SQLite

- journal_mode = WAL (concorrência)
- synchronous = NORMAL (balanço segurança/performance)
- busy_timeout = 5000 (espera 5s em lock)
- foreign_keys = ON (integridade referencial)
- temp_store = MEMORY (tabelas temp em RAM)
- cache_size = -20000 (~20MB cache)



■■■ Decisões Técnicas Críticas

8 decisões arquiteturais fundamentais

1. **SQLite com WAL:** Banco embarcado com excelente concorrência, sem servidor externo
2. **Backend Separado:** Processo Node.js isolado para crash isolation e recursos
3. **Downloads Verificáveis:** Instalador ~50MB com nodes baixados no primeiro uso (SHA256 + manifest assinado)
4. **API Boltz:** Provider externo com verificação local de scripts HTLC
5. **Taproot + MuSig2:** Key-path eficiente com fallback script-path + persistência de sessão
6. **PIN + Argon2id:** Criptografia user-friendly com device secret opcional (v1.1)
7. **BIP85 Swap Keys:** Child seed separado para recovery de swaps pendentes
8. **Config Management:** Tabela app_config com snapshot em cada swap

Raiz de Confiança: Os checksums são obtidos de um manifest assinado; a aplicação valida a assinatura com chave pública fixa (pinned) embutida no código. Sem servidor central de custódia/execução de swaps; apenas endpoints públicos de distribuição (GitHub Releases).

■ Stack Tecnológico

Tecnologias modernas para 2026

Componente	Tecnologia	Versão
Desktop	Electron	28+
Frontend	React + Vite	18 + 5
Linguagem	TypeScript	5.3
Database	SQLite + better-sqlite3	3.31+ + 9.2
Bitcoin	bitcoinjs-lib + Taproot	6.1
HD Wallet	bip32 + bip39	4.0 + 3.1
Crypto	argon2 + AES-GCM	0.31
LN Client	@grpc/grpc-js	1.9

Nota sobre Swaps LN↔BTC: O MVP suporta Submarine (Liquid→LN) e Reverse (LN→Liquid). Swaps diretos LN↔BTC (sem Liquid) são suportados via Submarine/Reverse na chain BTC se o par estiver disponível na API Boltz v2. Chain Swap (BTC↔Liquid) é implementado como swap atômico cross-chain.

■ Roadmap de Implementação

5 semanas até MVP completo

Fase	Duração	Entregas
Backend Core	2 semanas	Vault, Adapters, Boltz Client, Engine
Frontend	1.5 semanas	React UI, Onboarding, Dashboard, Swaps
Electron	1 semana	Main Process, IPC, Auto-update
Packaging	0.5 semana	Instaladores, Signing, Testes E2E

Status Atual (15% Completo)

- ■ Schema SQLite production-ready
- ■ DB Layer com CAS/transações
- ■ Node Manager com downloads verificáveis
- ■ 8 decisões técnicas documentadas
- ■ Key Vault (próximo)
- ■ Chain Adapters (próximo)
- ■ Boltz Client + MuSig2 (próximo)